

MAXDOP and CTP (Cost Threshold for Parallelism) in SQL Server, with **real-time performance tuning scenarios** using **AdventureWorks 2019** as an example.

◆ 1. What is MAXDOP (Maximum Degree of Parallelism)?

- **Definition:** MAXDOP controls how many CPU cores SQL Server can use in parallel to execute a single query.
- SQL Server uses parallelism when:
 - Query cost exceeds the **Cost Threshold for Parallelism (CTP)**.
 - The optimizer decides the query can benefit from parallel execution (scanning, joins, aggregations).
- **Default:** SQL Server sets it to 0 (meaning use all available CPUs).
- **Risk:**
 - If set too high → CPU thrashing, excessive context switching.
 - If set too low (like 1) → long-running queries that could have benefited from parallelism may run slowly.

◆ 2. What is CTP (Cost Threshold for Parallelism)?

- **Definition:** CTP sets the minimum estimated cost for a query to qualify for parallel execution.
- **Default = 5** (which is too low for modern hardware → almost everything qualifies for parallelism).
- **Estimated Cost** = Number generated by optimizer (not seconds, but an abstract measure).
- **Best Practice:** Usually increased to **25–50** depending on workload.

◆ 3. How MAXDOP & CTP Work Together

- CTP decides **if parallelism is even considered**.
- MAXDOP decides **how many CPUs are allocated** if parallelism is used.

For example:

- CTP = 5, MAXDOP = 0 → almost every medium query will go parallel on all CPUs.
- CTP = 50, MAXDOP = 4 → only expensive queries will use parallelism, and limited to 4 CPUs.

📦 Real-Time Examples Using AdventureWorks2019

Example 1: Default Settings (MAXDOP = 0, CTP = 5)

```
USE AdventureWorks2019;
GO
SET STATISTICS TIME, IO ON;
-- Example query: Large Aggregation (Sales Data)
SELECT SalesOrderID, SUM(LineTotal) AS TotalAmount
FROM Sales.SalesOrderDetail
GROUP BY SalesOrderID
ORDER BY TotalAmount DESC;
```

- **What Happens?**
 - Cost of query is **well above 5**, so it qualifies for parallelism.
 - MAXDOP = 0 → SQL Server may spread this query across all available CPUs.
 - On a server with 16 cores, you may see a **parallel plan** using many threads.
 - If multiple users run similar queries → **high CPU contention**.

Example 2: Limiting MAXDOP

-- Force MAXDOP to 4 (best practice for OLTP systems with many users)

```
ALTER DATABASE SCOPED CONFIGURATION SET MAXDOP = 4;
```

```
GO
```

Re-run the same query:

- Now SQL Server uses **only 4 CPUs** for parallel execution.
- Benefits:
 - Still faster than serial execution.
 - Prevents one query from hogging all CPUs.
- Drawback:
 - Might not be optimal for very large reporting queries.

Example 3: Adjusting CTP

-- Raise CTP so only expensive queries use parallelism

```
EXEC sp_configure 'show advanced options', 1;
```

```
RECONFIGURE;
```

```
EXEC sp_configure 'cost threshold for parallelism', 50;
```

```
RECONFIGURE;
```

Re-run the same query:

- Since this query has a high estimated cost (let's say >100), it **still goes parallel**.
- But smaller queries (like fetching Customer data with a join) **will now run serially**, saving CPU.

Example:

```
SELECT c.CustomerID, COUNT(*)
```

```
FROM Sales.SalesOrderHeader soh
```

```
JOIN Sales.Customer c ON soh.CustomerID = c.CustomerID
```

```
GROUP BY c.CustomerID;
```

- With CTP=5 → parallel plan.
- With CTP=50 → serial plan (faster overall because overhead of parallelism avoided).

Example 4: Per-Query MAXDOP Hint

Sometimes you want to override server settings **just for one query**:

```
SELECT SalesOrderID, SUM(LineTotal) AS TotalAmount
```

```
FROM Sales.SalesOrderDetail
```

```
GROUP BY SalesOrderID
```

```
ORDER BY TotalAmount DESC
```

```
OPTION (MAXDOP 2);
```

- Forces this query to use **2 CPUs only**, regardless of global setting.
- Useful when a query runs too aggressively in parallel, affecting other workloads.

Real-Time Scenarios

Scenario 1: OLTP System (Many Small Transactions)

- Defaults (MAXDOP=0, CTP=5) → even small queries go parallel → CPU saturation.
- **Fix:**
 - MAXDOP = 4 (per NUMA node best practice).
 - CTP = 50 → prevents small OLTP queries from using parallelism.
- **Result:** Faster transactions, lower CPU utilization.

Scenario 2: Reporting Queries on AdventureWorks

- Large **aggregations, joins, and reports** benefit from parallelism.
- MAXDOP=1 (forced serial) would make queries very slow.
- **Fix:**
 - MAXDOP = 4 or 8 → balanced parallelism.
 - CTP = 25 → only bigger queries use parallelism.
- **Result:** Complex reporting queries finish much faster without hurting OLTP workloads.

Scenario 3: ETL / Data Warehouse Load

- Bulk loads or aggregations (like fact table loads).
- Benefit from **higher MAXDOP** (like 8 or even default).
- CTP can stay moderate (25–50).
- **Example:** Loading FactInternetSales into warehouse from AdventureWorksDW.

```
INSERT INTO FactInternetSales_Summary (ProductKey, TotalSales)
```

```
SELECT ProductKey, SUM(SalesAmount)
```

```
FROM FactInternetSales
```

```
GROUP BY ProductKey
```

```
OPTION (MAXDOP 8);
```

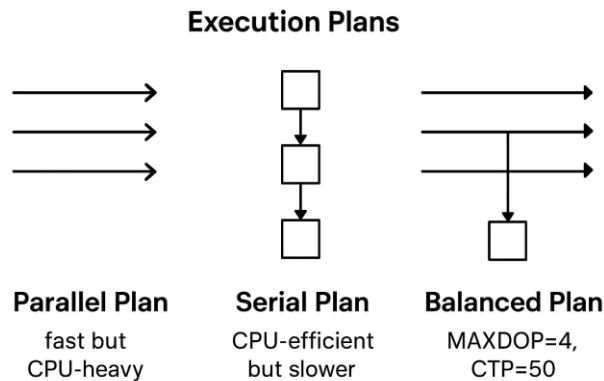
- Parallelism speeds up the aggregation process significantly.

Best Practices (Industry + Microsoft Guidelines)

1. **Set MAXDOP at Instance Level:**
 - OLTP: 4–8
 - OLAP/Reporting: higher (depends on CPU count).
 - NUMA awareness → don't cross NUMA node boundaries.
2. **Raise CTP from default 5:**
 - Typically 25–50 for OLTP.
 - Tune based on workload (test!).
3. **Use Query Hints for Exceptions:**
 - OPTION (MAXDOP N) for special cases.
4. **Monitor with DMVs:**
5. `SELECT *`
6. `FROM sys.dm_exec_query_stats`
7. `CROSS APPLY sys.dm_exec_query_plan(plan_handle)`
8. `WHERE query_plan LIKE '%Parallelism%';`

✓ In summary:

- **MAXDOP** controls **how wide** parallelism goes.
- **CTP** controls **when** parallelism kicks in.
- Tuning both together can **drastically improve performance** by ensuring small OLTP queries don't consume CPUs unnecessarily, while large queries still benefit.



Step-by-step execution plan comparison using **AdventureWorks2019** so you can clearly see the difference between **serial vs parallel plans** and how **MAXDOP + CTP** tuning affects performance.

⚡ Execution Plan Comparison – Serial vs Parallel

We'll use a **large aggregation query** on AdventureWorks2019 (which has enough rows in Sales.SalesOrderDetail to trigger parallelism).

Step 1: Check Current Settings

-- Show current configuration

```
EXEC sp_configure 'show advanced options', 1;  
RECONFIGURE;
```

```
EXEC sp_configure 'max degree of parallelism';  
EXEC sp_configure 'cost threshold for parallelism';
```

👉 Usually you'll see:

- MAXDOP = 0
- CTP = 5

This means: **parallelism is wide open, even for small queries.**

Step 2: Example Query

```
USE AdventureWorks2019;  
GO
```

```
SET STATISTICS TIME, IO ON;
```

```
SELECT SalesOrderID, SUM(LineTotal) AS TotalAmount  
FROM Sales.SalesOrderDetail  
GROUP BY SalesOrderID  
ORDER BY TotalAmount DESC;
```

Step 3: Execution Plan with Default (MAXDOP = 0, CTP = 5)

- Open **Actual Execution Plan** in SSMS (Ctrl+M before running query).
- You'll see:
 - **Parallelism operators** (yellow arrows) such as:
 - Parallelism (Distribute Streams)
 - Parallelism (Repartition Streams)
 - Parallelism (Gather Streams)

- Each operator spreads work across multiple threads.
- **Performance Metrics:**
 - **CPU Time is high** because many threads are used.
 - **Elapsed Time is low** (query finishes quickly).
 - If multiple users run this → CPU saturation.

Step 4: Force Serial Plan (MAXDOP = 1)

```
ALTER DATABASE SCOPED CONFIGURATION SET MAXDOP = 1;  
GO
```

Re-run the query with SET STATISTICS TIME ON.

- Execution Plan now shows:
 - **No parallelism operators** (single thread).
 - Hash Match and Aggregate run in **serial mode**.
- **Performance Metrics:**
 - **CPU Time is low** (only 1 thread).
 - **Elapsed Time is higher** (query runs longer).
- Scenario: safer for OLTP workloads but reporting queries become slower.

Step 5: Balanced Parallelism (MAXDOP = 4, CTP = 50)

```
ALTER DATABASE SCOPED CONFIGURATION SET MAXDOP = 4;  
EXEC sp_configure 'cost threshold for parallelism', 50;  
RECONFIGURE;
```

Re-run the query.

- Execution Plan:
 - Still parallel, but only **4 CPUs used instead of all**.
 - Parallelism operators remain, but reduced CPU usage.
- **Performance Metrics:**
 - **CPU Time moderate**.
 - **Elapsed Time still low**.
 - Best balance: query runs fast without exhausting all CPUs.

Real-Time Comparison (Hypothetical Metrics)

Setting	CPU Time (ms)	Elapsed Time (ms)	Plan Operators
MAXDOP=0, CTP=5	9000	1500	Full parallel (all cores)
MAXDOP=1	2000	8000	Serial only
MAXDOP=4, CTP=50	4000	2000	Balanced parallel

(Values are sample-like; exact numbers depend on your CPU count & data volume.)

How to Demonstrate in Interviews / Presentations

1. Run the same query three times with different settings.

2. **Capture execution plans** in SSMS.
3. Highlight:
 - Parallelism operators present or not.
 - Differences in CPU time vs elapsed time.
 - Explain trade-off: OLTP vs OLAP.

 Takeaway:

- **Parallel plans** = fast but CPU-heavy.
- **Serial plans** = CPU-efficient but slower.
- **Balanced (MAXDOP=4, CTP=50)** = the sweet spot for most workloads.